

Implementation Plan: Membership, Authentication, and Librarian Permissions

Step 2 of the Library Architecture Implementation

Target: Step 2 (Membership & Auth) + Step 1 (Database Foundation)	Environment: Python 3.11 / Flask	Database: SQLite3	Status: Ready for Review
--	---	--------------------------	---------------------------------

1. Overview & Architecture

This plan covers the implementation of member registration, password hashing, authentication, role-based librarian permissions, and the foundational SQLite database operations required for persistence.

According to the project implementation guide (`README.md`):

- Step 1:** Review `schema.sql`, implement `db.py`, and initialize the database.
- Step 2:** Implement membership and authentication, including password hashing and librarian permissions.

Because member registration and authentication require persistent SQLite storage, this plan combines the database foundation (`db.py`, connection lifecycle, schema initialization) with the membership and authentication features so that all functionality can be immediately executed and verified.

User Review & Architectural Highlights

- Authentication Mechanism:** Standard Flask session cookies (built into Flask with zero external dependencies) combined with route decorators `@login_required` and `@librarian_required`.
- Role Storage & Separation:** Adding `role TEXT NOT NULL DEFAULT 'member' CHECK(role IN ('member', 'librarian'))` to the `member` table in `schema.sql` to differentiate standard borrowers from librarians without creating unnecessary schema complexity.
- Security:** All passwords hashed using standard PBKDF2/SHA256 via Werkzeug (`generate_password_hash` / `check_password_hash`). Password hashes are never exposed in responses or API payloads.

2. Proposed Component Changes

A. Database Foundation

MODIFY `app/schema.sql`

- Add `role TEXT NOT NULL DEFAULT 'member' CHECK(role IN ('member', 'librarian'))` to the `member` table definition.
- Ensure foreign key constraints (`PRAGMA foreign_keys = ON;`) and table structures are configured cleanly.

MODIFY `app/db.py`

- `get_db()` : Provides a request-scoped SQLite connection stored in `flask.g` with `row_factory = sqlite3.Row` and foreign keys enabled.
- `close_db(error=None)` : Cleans up and closes the connection during Flask context teardown.
- `init_db()` : Creates the application instance directory and executes `schema.sql` against the database.
- `init_app(app)` : Registers teardown handler and provides CLI command `flask init-db`.

MODIFY `app/__init__.py`

- Call `init_app(app)` from `app.db`.
- Configure `SECRET_KEY` for secure session cookie signing.
- Ensure instance path directory exists.

B. Data Models & Service Layer

MODIFY `app/models.py`

- Update `Member` dataclass with `role: str = "member"`.
- Add serialization helper (e.g. `to_dict(include_password=False)`) to ensure credentials are never leaked.

MODIFY `app/services/members.py`

- `register_member(full_name, username, password, role="member")` : Validates inputs (non-empty, minimum length), hashes password using `generate_password_hash` , records ISO 8601 UTC timestamp, catches duplicate username (`sqlite3.IntegrityError`) and raises descriptive `ValueError` , returns `Member` instance.
- `authenticate_member(username, password)` : Retrieves member by username, verifies password with `check_password_hash` , returns `Member` instance or `None` on invalid credentials.
- `get_member(member_id)` : Look up a registered member by ID.
- `get_member_by_username(username)` : Look up a member by username.
- `is_librarian(member)` : Utility to evaluate whether a member holds librarian privileges.

C. API Routes & Authentication Middleware

MODIFY `app/routes.py`

- Add decorators:
 - `@login_required` : Verifies active session or returns `401 Unauthorized` .
 - `@librarian_required` : Verifies logged-in user is a librarian or returns `403 Forbidden` .
- Add endpoints:
 - `POST /api/auth/register` : Registers a new member/librarian (returns 201 Created).
 - `POST /api/auth/login` : Validates credentials and sets session cookie (returns 200 OK).
 - `POST /api/auth/logout` : Clears user session (returns 200 OK).
 - `GET /api/auth/me` : Retrieves current authenticated member profile.
 - `GET /api/members/<int:member_id>` : Member profile lookup (protected).

D. Automated Test Suites

NEW `tests/test_db.py`

- Test database connection lifecycle, foreign key enforcement, and clean table initialization.

NEW `tests/test_members.py`

- Test valid member and librarian registration.
- Test duplicate username rejection with appropriate error.
- Test password hashing verification (hash is not raw password).
- Test authentication success and failure branches.
- Test librarian permission detection.

NEW**tests/test_auth_routes.py**

- Test HTTP registration endpoint with input validation.
- Test HTTP login and session persistence.
- Test HTTP logout session invalidation.
- Test access control on protected routes (401 for unauthenticated, 403 for non-librarian).

3. Verification Plan

A. Automated Testing

Run the test suite using Python's standard test runner or pytest:

```
# Standard test discovery (zero additional dependencies needed)
python -m unittest discover tests -v

# Or using pytest
pytest -v
```

B. Manual API Verification

Commands to verify end-to-end functionality via Flask CLI:

```
# 1. Initialize SQLite database
flask --app app init-db

# 2. Start local development server
flask --app app run --debug

# 3. Test API Endpoints:
# - Register: POST /api/auth/register
# - Login:    POST /api/auth/login
# - Check:    GET  /api/auth/me
# - Logout:   POST /api/auth/logout
```